

BAHRIA UNIVERSITY KARACHI CAMPUS

DEPARTMENT OF COMPUTER SCIENCE



Bahria University
Discovering Knowledge

Artificial Intelligence Lab (CSL-411) BS CS

(6A) – Spring`25

PROJECT NAME:

**Modify Spotify Music Recommendation System using Face
Expressions**

Submitted by:

Alishba Siraj (02-134222-005)

Syeda Sarah Batool Naqvi (02-134222-011)

Nahin Fatima (02-134222-102)

Submitted To:

Engr. Rabia Amjad

Submitted On:

2nd/ June /2025

Acknowledgement

We would like to express our gratitude to our course Lab instructor Miss Rabia Amjad and course instructor Sir Tariq for their continuous guidance and support. Also thankful to Open Source contributors of DeepFace and OpenCV libraries, which made this project possible. We are also grateful for the collaborative discussions and encouragement from our peers.

Abstract

Moodify is a real-time AI-based emotion recognition system that leverages facial analysis to detect a user's mood via webcam and recommend appropriate songs from a dataset found on kaggle. The system uses DeepFace for emotion detection, OpenCV for real-time camera integration, and Python libraries for emotion-to-mood mapping. Unlike traditional recommendation systems, Moodify reacts to live user input without requiring manual mood input, offering a personalized and adaptive experience.

Table of Contents

Acknowledgement.....	2
Abstract	2
Introduction	4
Problem Statement.....	4
Existing Systems	4
Objectives and Goals	4
Project Scope.....	5
Workflow	5
Overview of Project.....	5
Tools and Technologies.....	5
Project Features	6
Functional Requirements.....	6
Non Funtional Requirements.....	6
AI & Data Science Concepts (Course Concepts Implemented).....	6
Output.....	7
Future Work	25
Conclusion.....	25
References	25

Introduction

With the rapid integration of AI into everyday life, emotion-aware systems have become increasingly relevant. Moodify aims to bring emotional intelligence to music recommendation by detecting facial expressions and suggesting songs based on the user's real-time mood. This system bridges the gap between user emotion and music personalization.

Problem Statement

Most music recommendation systems are based on past behavior, user ratings, or manual input of mood. There is a lack of adaptive, real-time systems that can recommend music based on the user's live emotional state. Moodify addresses this by using computer vision and deep learning to capture facial expressions and generate mood-appropriate song suggestions.

Existing Systems

System	Features	Lacks
Spotify	Huge music library, curated playlists	No real-time emotion adaptation
Moodify clone apps (like EmoPlayer)	Emotion detection + playlist	Most are not real-time or accurate
YouTube Music	Context-aware playlists	No face-based mood detection

Difference: Moodify focuses on *real-time facial recognition*, not behavior history or keywords.

Objectives and Goals

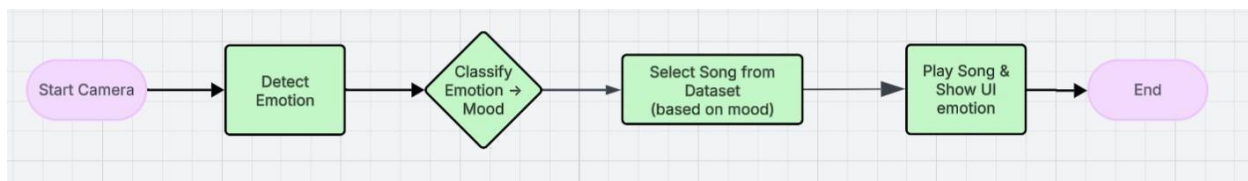
- Implement real-time webcam-based emotion recognition.
- Link detected emotions to mood categories.
- Play songs locally using Spotify-sourced metadata.
- Design a visually appealing UI using OpenCV and custom themes.
- Create a modular system ready for further improvements.

Project Scope

The project focuses on:

- Real-time facial emotion detection
- Mood classification based on facial emotions
- Music recommendation system based on moods
- Local song dataset from kaggle and integration with Spotify
- Modular, extendable codebase for future AI enhancements

Workflow



Overview of Project

Moodify is an AI-powered multimedia music player that detects human emotions in real time through webcam input and plays mood-aligned songs from a locally stored Spotify-derived dataset. Using DeepFace for facial emotion detection, it maps detected emotions (e.g., happy, sad, neutral) to mood categories (happy, sad, calm, energetic) and plays music accordingly via browser-based playback. A custom OpenCV interface enriches the user experience with live visual feedback, displaying the current emotion, mood, and track metadata using a stylish, wave-themed UI. To inform and support mood classification, extensive Exploratory Data Analysis (EDA) was performed on the song dataset. This includes mood vs. feature heatmaps, audio feature correlation matrices, boxplots comparing energy across moods (styled like equalizer bars), pairplots showing inter-feature relationships, radar charts visualizing the “audio signature” of top tracks per mood, and a pulse-style duration distribution plot. These visualizations helped uncover feature patterns tied to specific moods, ensuring both technical soundness and an engaging user experience.

Tools and Technologies

- **Programming Language:** Python
- **Libraries & Frameworks:** OpenCV, DeepFace, seaborn, matplotlib, pandas, NumPy, VLC (optional), playsound
- **Visualization:** seaborn, matplotlib
- **IDE:** Jupyter Notebook
- **Data Source:** Custom-curated Spotify dataset (CSV) with audio features from Kaggle
- **Interface:** Custom GUI with OpenCV (wave animation, mood circle, emotion tracking)

Project Features

- **Real-Time Emotion Detection:** Uses webcam feed and DeepFace to detect facial emotions.
- **Mood Mapping:** Maps emotions (happy, sad, angry, surprise, fear) to moods (happy, calm, energetic, sad).
- **Automated Music Playback:** Plays mood-aligned songs from a local Spotify-sourced dataset using system browser.
- **Custom UI Display:** Shows live emotion, mood name, and currently playing track info with wave-style animation.
- **Data Visualization:** Includes heatmaps, radar charts, boxplots, pairplots, and duration distributions to analyze song features.

Functional Requirements:

- System must capture real-time video input through webcam.
- System must detect emotion using DeepFace.
- System must map emotions to moods and select a song accordingly.
- System must display the current mood, emotion, and song info on a custom GUI.
- System must visualize dataset insights via EDA plots.

Non-Functional Requirements:

- **Responsiveness:** The system should process and respond within a few seconds.
- **Scalability:** Can support additional moods or larger datasets with minimal changes.
- **Maintainability:** Modular code for easy updates or model enhancements.
- **User Experience:** Visually engaging interface with smooth mood-to-music transition.
- **Accuracy:** Relies on DeepFace's pre-trained models for reliable emotion recognition.

AI & Data Science Concepts (Course Concepts Implemented)

- **Computer Vision:** OpenCV for live video feed processing
- **Deep Learning:** DeepFace library for facial emotion recognition
- **Data Handling:** Pandas used to process and filter song CSVs
- **Classification:** Emotions are categorized into moods
- **Real-time Application:** Integration of inference loop with GUI

Output

Imports

```
[1]: import cv2
import time
from deepface import DeepFace
import pandas as pd
import webbrowser
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import MinMaxScaler
```

WARNING:tensorflow:From C:\Users\ESHOP\anaconda3\Lib\site-packages\tf_keras\src\losses.py:2976: The name tf.losses.sparse_softmax_cross_entropy is deprecated. Please use tf.compat.v1.losses.sparse_softmax_cross_entropy instead.

```
[2]: df = pd.read_csv('C:/Users/ESHOP/Downloads/mood_song.csv')
```

EDA (Exploratory Data Analysis)

```
[3]: df.head()
```

```
[3]:
```

	Unnamed: 0.1	Unnamed: 0	duration (ms)	danceability	energy	loudness	speechiness	acousticness	instrumentalness	liveness	valence	tempo	spec_rate	labels
0	0	0	195000.0	0.611	0.614	-8.815	0.0672	0.0169	0.000794	0.7530	0.520	128.050	3.446154e-07	2 spotify:tr
1	1	1	194641.0	0.638	0.781	-6.848	0.0285	0.0118	0.009530	0.3490	0.250	122.985	1.464234e-07	1 spotify:tra
2	2	2	217573.0	0.560	0.810	-8.029	0.0872	0.0071	0.000008	0.2410	0.247	170.044	4.007850e-07	1 spotify:tr
3	3	3	443478.0	0.525	0.699	-4.571	0.0353	0.0178	0.000088	0.0888	0.199	92.011	7.959809e-08	0 spotify:t
4	4	4	225862.0	0.367	0.771	-5.863	0.1060	0.3650	0.000001	0.0965	0.163	115.917	4.693131e-07	1 spotify

```
[4]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 277938 entries, 0 to 277937
Data columns (total 15 columns):
#   Column              Non-Null Count  Dtype
---  -
0   Unnamed: 0.1         277938 non-null  int64
1   Unnamed: 0           277938 non-null  int64
2   duration (ms)        277938 non-null  float64
3   danceability         277938 non-null  float64
4   energy               277938 non-null  float64
5   loudness             277938 non-null  float64
6   speechiness          277938 non-null  float64
7   acousticness         277938 non-null  float64
8   instrumentalness     277938 non-null  float64
9   liveness             277938 non-null  float64
10  valence              277938 non-null  float64
11  tempo                277938 non-null  float64
12  spec_rate            277938 non-null  float64
13  labels               277938 non-null  int64
14  uri                  277938 non-null  object
dtypes: float64(11), int64(3), object(1)
memory usage: 31.8+ MB
```

```
[9]: df.isnull().sum()
```

```
[9]: Unnamed: 0.1      0
     Unnamed: 0      0
     duration (ms)   0
     danceability    0
     energy          0
     loudness        0
     speechiness     0
     acousticness    0
     instrumentalness 0
     liveness        0
     valence         0
     tempo           0
     spec_rate       0
     labels          0
     uri             0
     dtype: int64
```

```
[6]: df.size
```

```
[6]: 4169070
```

```
[7]: df.shape
```

```
[7]: (277938, 15)
```

```
[8]: df.columns
```

```
[8]: Index(['Unnamed: 0.1', 'Unnamed: 0', 'duration (ms)', 'danceability', 'energy',
        'loudness', 'speechiness', 'acousticness', 'instrumentalness',
        'liveness', 'valence', 'tempo', 'spec_rate', 'labels', 'uri'],
        dtype='object')
```

```
[5]: df.describe()
```

```
[5]:
```

	Unnamed: 0.1	Unnamed: 0	duration (ms)	danceability	energy	loudness	speechiness	acousticness	instrumentalness	liveness
count	277938.000000	277938.000000	2.779380e+05	277938.000000	277938.000000	277938.000000	277938.000000	277938.000000	277938.000000	277938.000000
mean	138968.500000	138968.500000	2.324961e+05	0.552583	0.556866	-10.363654	0.087913	0.386583	0.255044	0.189217
std	80233.933896	80233.933896	1.171830e+05	0.188905	0.279681	6.672049	0.112500	0.364504	0.373745	0.163596
min	0.000000	0.000000	6.706000e+03	0.000000	0.000000	-60.000000	0.000000	0.000000	0.000000	0.000000
25%	69484.250000	69484.250000	1.720130e+05	0.431000	0.342000	-12.747000	0.035900	0.033800	0.000000	0.096200
50%	138968.500000	138968.500000	2.131055e+05	0.571000	0.591000	-8.397000	0.047100	0.262000	0.001090	0.121000
75%	208452.750000	208452.750000	2.648660e+05	0.693000	0.792000	-5.842000	0.082200	0.754000	0.645000	0.227000
max	277937.000000	277937.000000	3.919895e+06	0.989000	1.000000	4.882000	0.965000	0.996000	1.000000	1.000000

Data Preprocessing

```
[10]: df = df.drop(columns=['Unnamed: 0.1', 'Unnamed: 0'])

[11]: df.head()
```

	duration (ms)	danceability	energy	loudness	speechiness	acousticness	instrumentalness	liveness	valence	tempo	spec_rate	labels
0	195000.0	0.611	0.614	-8.815	0.0672	0.0169	0.000794	0.7530	0.520	128.050	3.446154e-07	2 spotify:track:3v6sBj3swihU8pXQQHl
1	194641.0	0.638	0.781	-6.848	0.0285	0.0118	0.009530	0.3490	0.250	122.985	1.464234e-07	1 spotify:track:7KCWmFdw0TzoJbKtqR
2	217573.0	0.560	0.810	-8.029	0.0872	0.0071	0.000008	0.2410	0.247	170.044	4.007850e-07	1 spotify:track:2CY92qejUrhYPUASawI
3	443478.0	0.525	0.699	-4.571	0.0353	0.0178	0.000088	0.0888	0.199	92.011	7.959809e-08	0 spotify:track:11BPfwVbB7vok7KfjBe
4	225862.0	0.367	0.771	-5.863	0.1060	0.3650	0.000001	0.0965	0.163	115.917	4.693131e-07	1 spotify:track:3yUJKPsvjvThlcQWTS

```

[13]: mood_map = {0: 'sad', 1: 'happy', 2: 'energetic', 3: 'calm'}
df['mood'] = df['labels'].map(mood_map)

[14]: df['playable_url'] = 'https://open.spotify.com/track/' + df['uri'].str.split(':')[0].str[-1]

```

Visualization

```
[17]: from matplotlib.colors import LinearSegmentedColormap

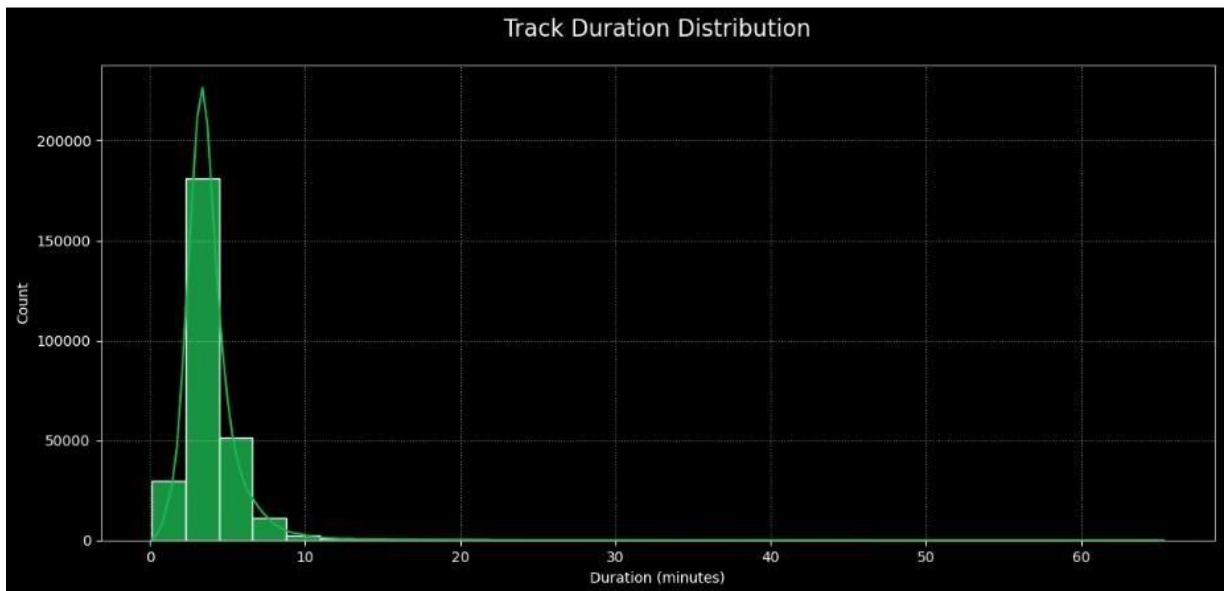
music_palette = ["#1DB954", "#191414", "#FFFFFF", "#B3B3B3", "#535353"]
spotify_gradient = LinearSegmentedColormap.from_list("spotify", ["#191414", "#1DB954"])

plt.style.use('dark_background')
sns.set_palette(music_palette)
plt.rcParams['font.family'] = 'DejaVu Sans'
plt.rcParams['axes.edgecolor'] = '#B3B3B3'

[18]: df['duration_min'] = df['duration (ms)'] / 60000
print(df['duration_min'].describe())
```

count	277938.000000
mean	3.874935
std	1.953050
min	0.111767
25%	2.866883
50%	3.551758
75%	4.414433
max	65.331583
Name: duration_min, dtype: float64	

```
[16]: # Duration Distribution (with pulse effect)
plt.figure(figsize=(14, 6))
ax = sns.histplot(df['duration (ms)']/60000, bins=30, kde=True, color='#1DB954', alpha=0.8)
plt.title('Track Duration Distribution', fontsize=16, pad=20)
plt.xlabel('Duration (minutes)')
plt.ylabel('Count')
ax.grid(True, linestyle=':', alpha=0.5)
plt.show()
```



```
from math import pi

moods = df['labels'].unique()
features = ['danceability', 'energy', 'loudness',
            'speechiness', 'acousticness', 'valence']
def plot_mood_radar(df, mood_label, top_n=3):
    mood_name = mood_map.get(mood_label, str(mood_label))
    subset = df[df['labels'] == mood_label].nlargest(top_n, 'energy')

    plt.figure(figsize=(10, 10))

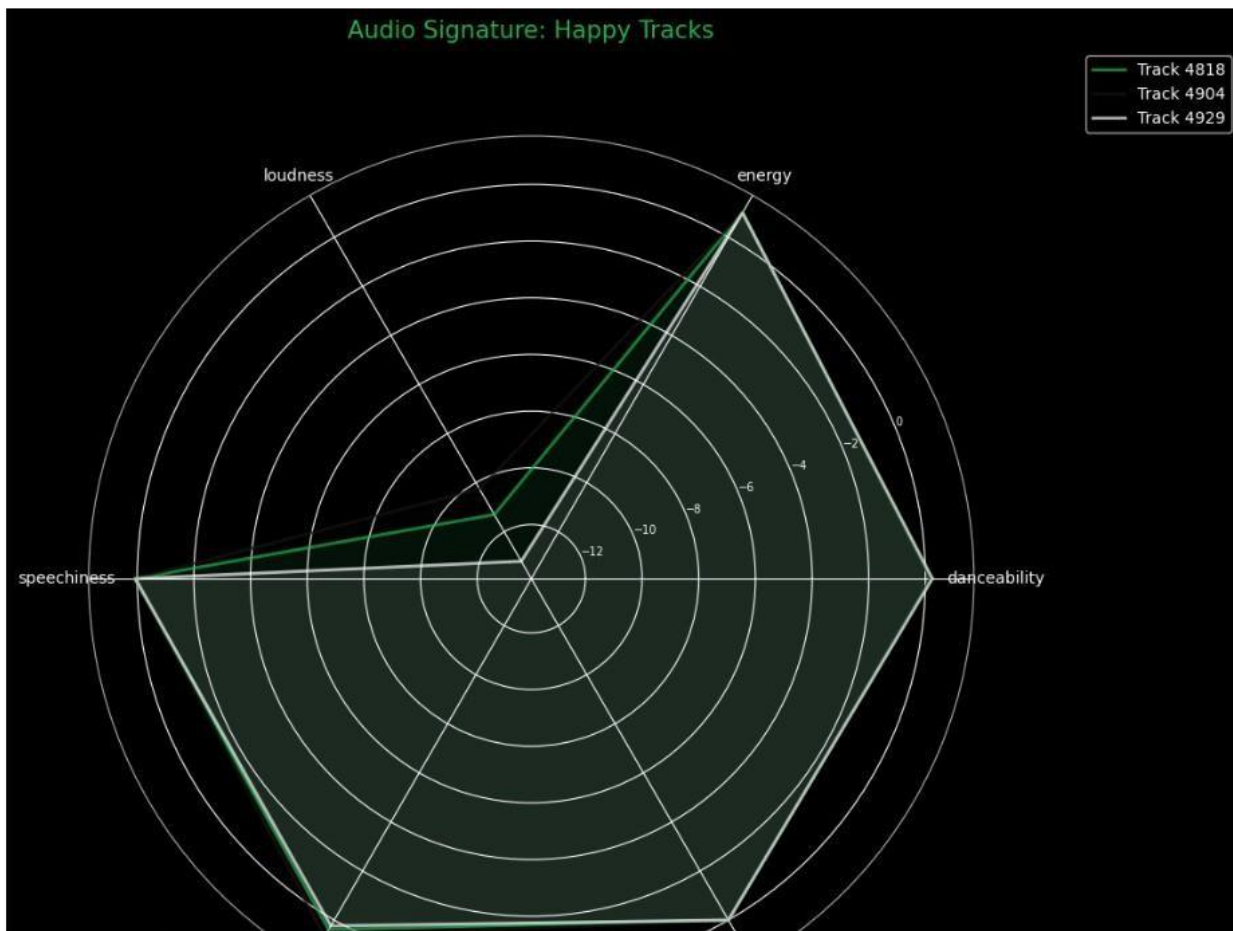
    for idx, row in subset.iterrows():
        values = row[features].tolist()
        values += values[:1]

        angles = [n / float(len(features)) * 2 * pi for n in range(len(features))]
        angles += angles[:1]

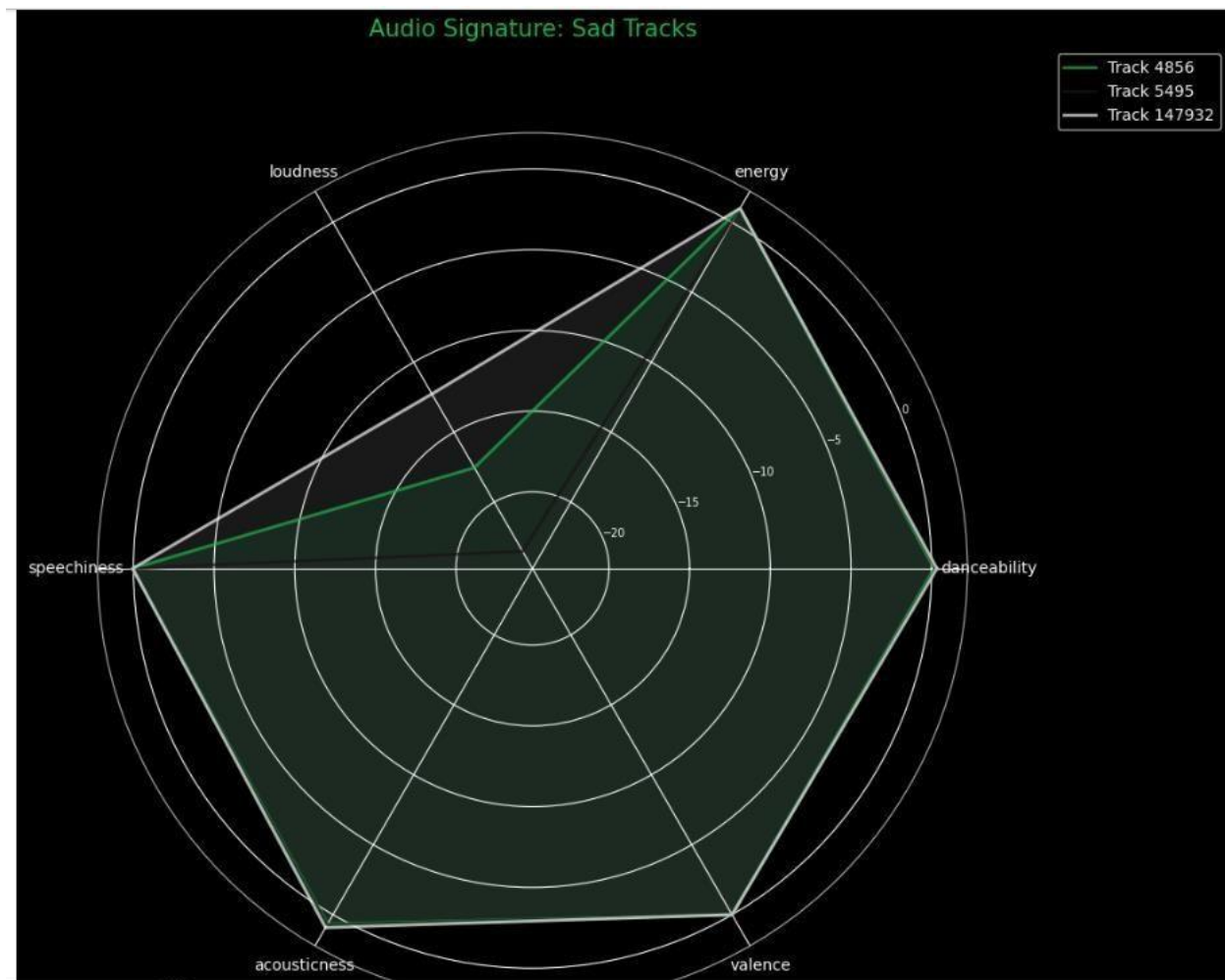
        ax = plt.subplot(111, polar=True)
        ax.plot(angles, values, linewidth=2, linestyle='solid', label=f"Track {idx}", alpha=0.7)
        ax.fill(angles, values, alpha=0.1)
```

```
plt.xticks(angles[:-1], features, color='white', size=10)
plt.yticks(color="white", size=7)
plt.title(f'Audio Signature: {mood_name.capitalize()} Tracks', size=15, y=1.1, color='#1DB954')
plt.legend(loc='upper right', bbox_to_anchor=(1.3, 1.1))
plt.show()
```

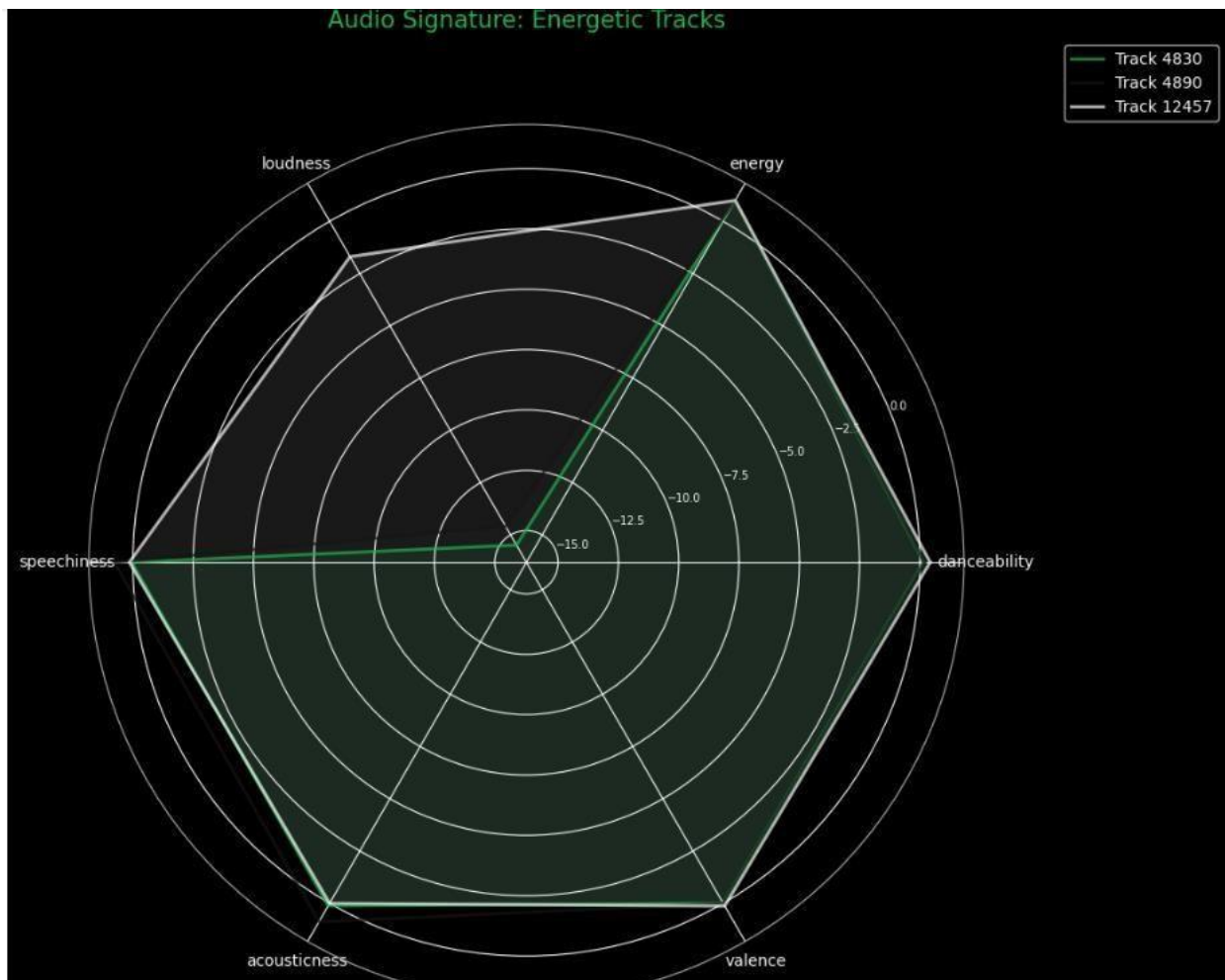
```
plot_mood_radar(df, mood_label=1)
```



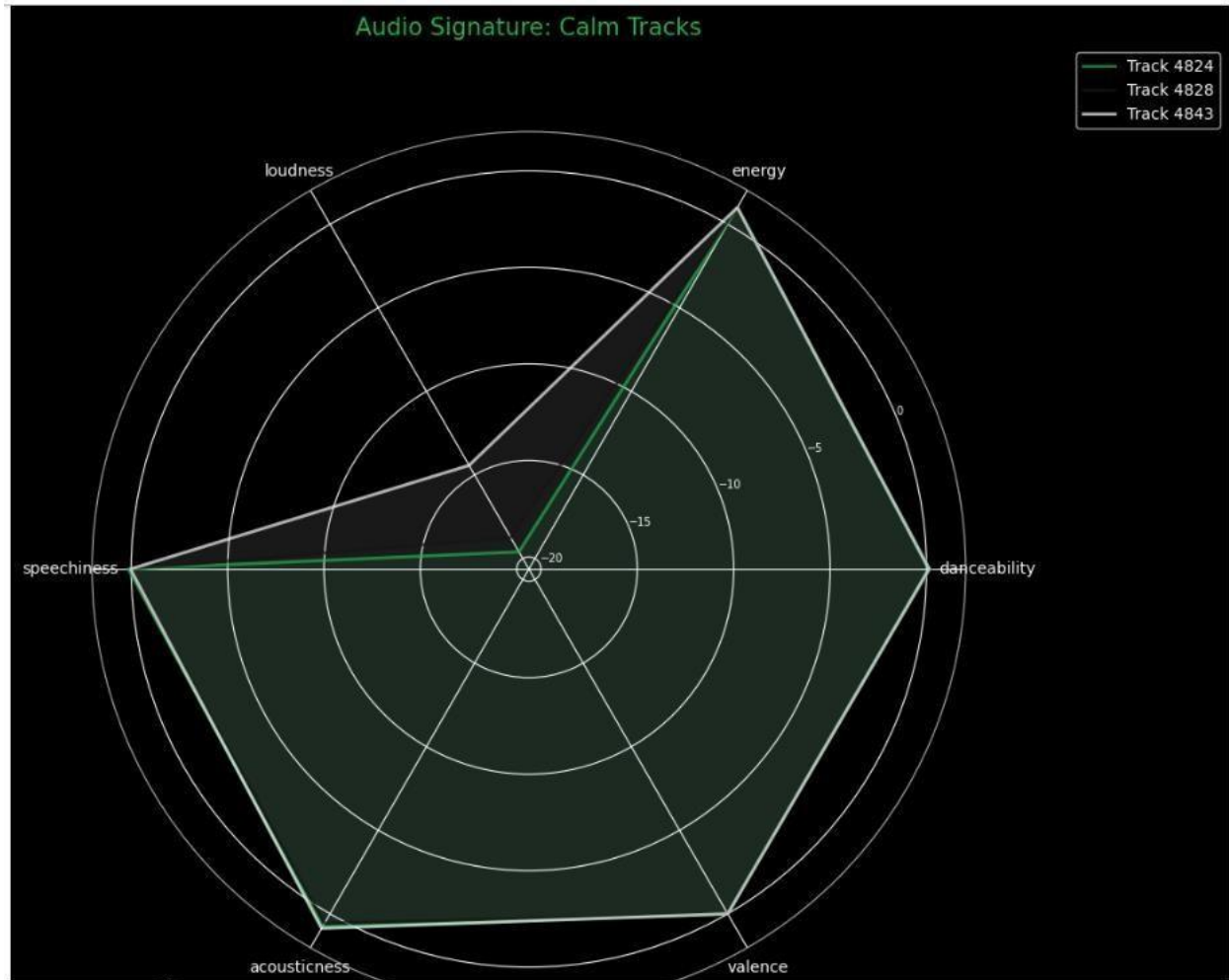
```
plot_mood_radar(df, mood_label=0)
```



```
plot_mood_radar(df, mood_label=2)
```



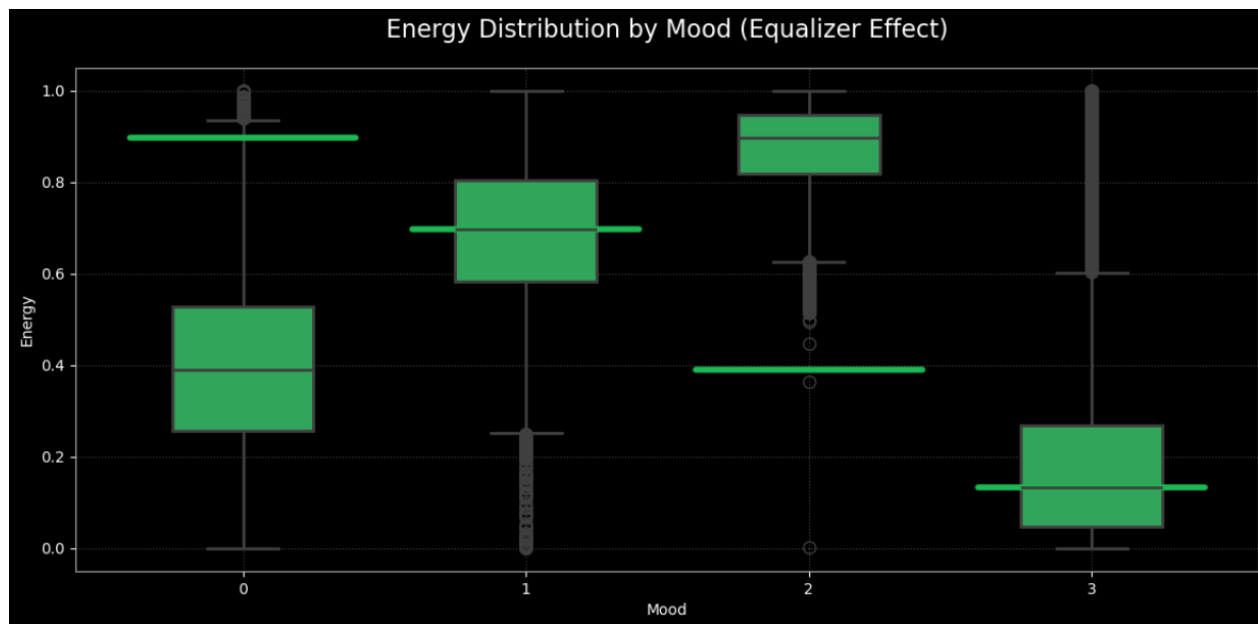
```
plot_mood_radar(df, mood_label=3)
```



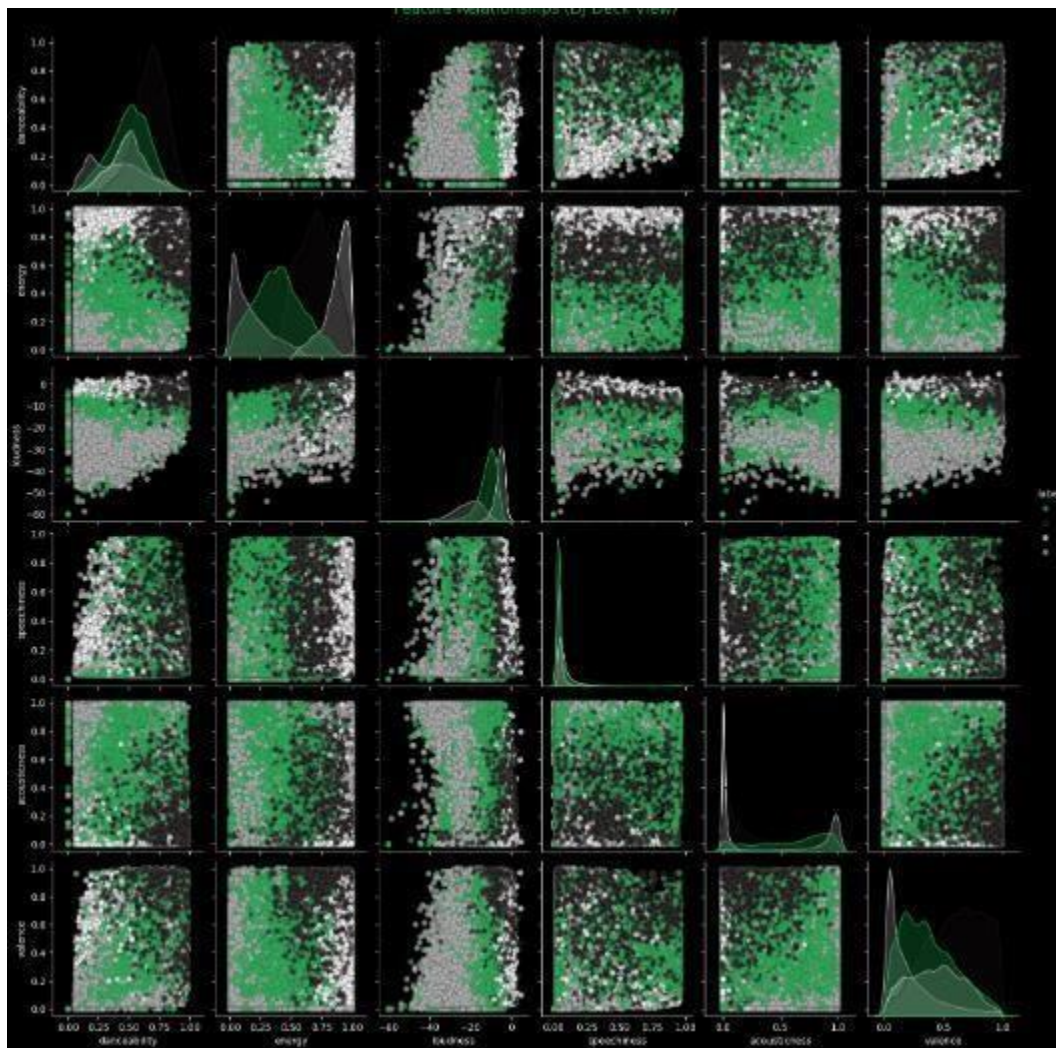
```
plt.figure(figsize=(14, 6))
ax = sns.boxplot(x='labels', y='energy', data=df, width=0.5, linewidth=2, flierprops=dict(marker='o', markersize=8))

# Add equalizer bars
for i, mood in enumerate(moods):
    median = df[df['labels'] == mood]['energy'].median()
    ax.plot([i-0.4, i+0.4], [median, median], color='#1DB954', linewidth=4, solid_capstyle='round')

plt.title('Energy Distribution by Mood (Equalizer Effect)',
          fontsize=16, pad=20)
plt.xlabel('Mood')
plt.ylabel('Energy')
ax.grid(True, linestyle=':', alpha=0.3)
plt.show()
```



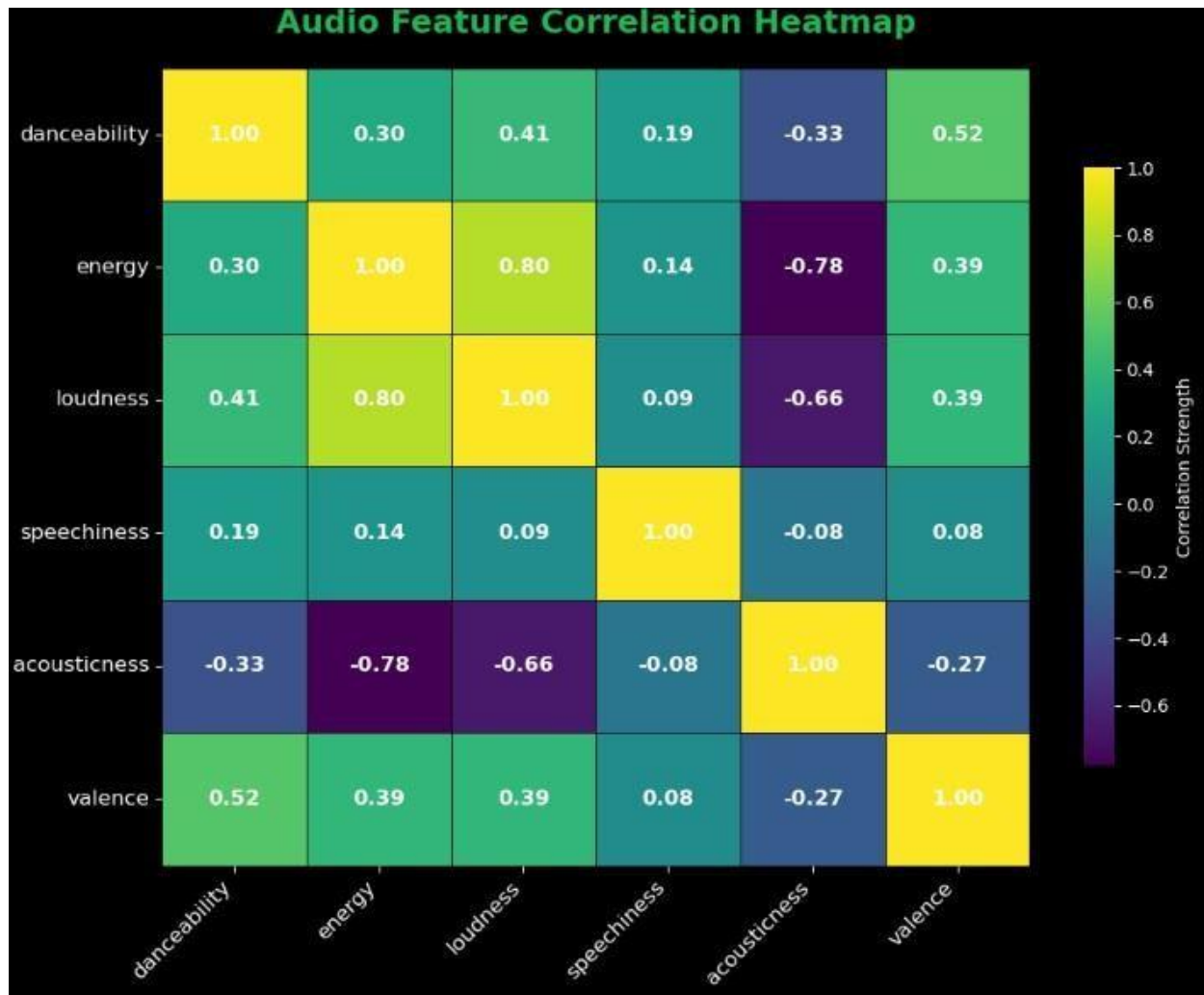
```
# Feature Pairplot (DJ Deck Style)
print("\nGenerating interactive pairplot...")
g = sns.pairplot(df[features + ['labels']],
                 hue='labels',
                 palette=music_palette,
                 plot_kws=dict(alpha=0.7, edgecolor='#535353'),
                 diag_kind='kde')
g.fig.suptitle('Feature Relationships (DJ Deck View)', y=1.02, fontsize=16, color='#1DB954')
plt.show()
```

```
# Compute correlation matrix
corr = df[features].corr()

# Set up figure
plt.figure(figsize=(10, 8))
sns.heatmap(
    corr,
    annot=True,
    fmt=".2f",
    cmap="viridis",
    linewidths=0.5,
    linecolor='#191414',
    cbar_kws={"shrink": .75, "label": "Correlation Strength"},
    annot_kws={"size": 12, "weight": "bold", "color": "white"}
)

plt.title("Audio Feature Correlation Heatmap", fontsize=18, color='#1D8954', weight='bold', pad=20)
plt.xticks(rotation=45, ha='right', fontsize=12, color='white')
plt.yticks(rotation=0, fontsize=12, color='white')
plt.tight_layout()
plt.show()
```

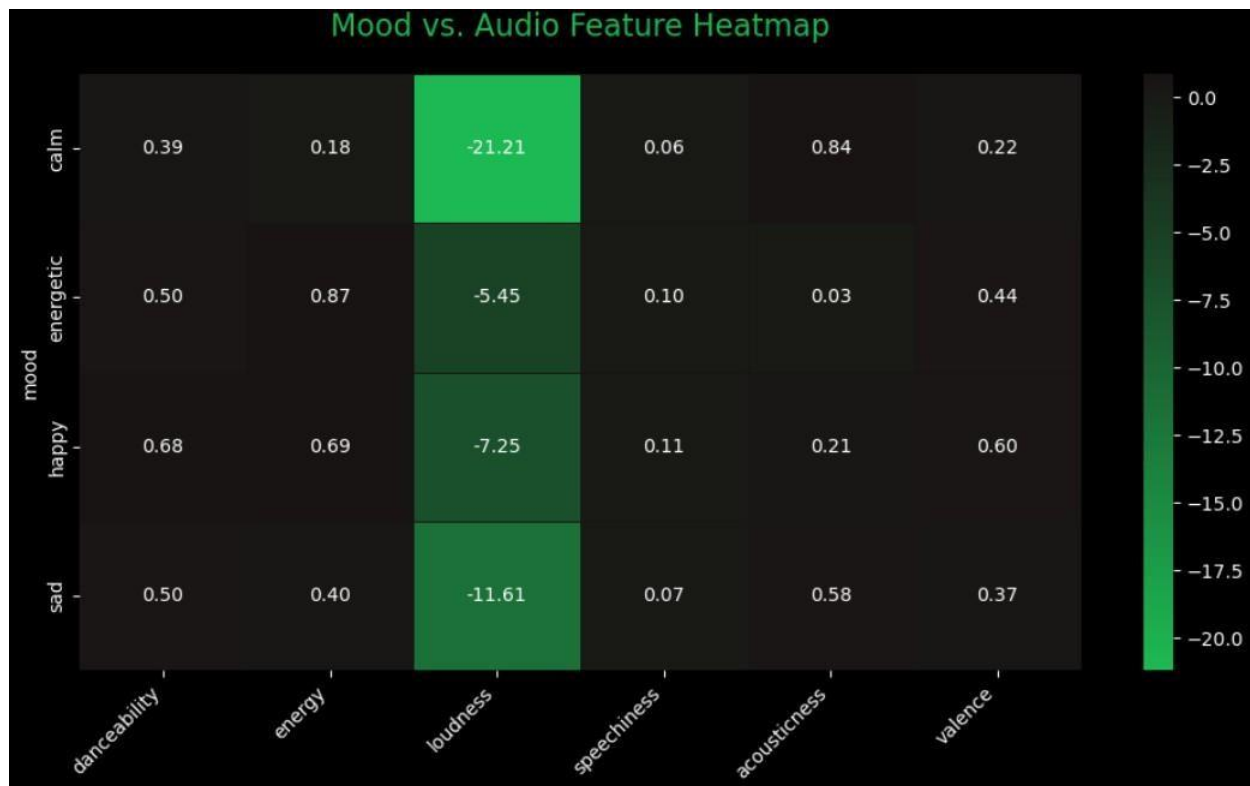



```
mood_feature_avg = df.groupby('mood')[features].mean().round(2)

spotify_cmap = LinearSegmentedColormap.from_list("spotify_gradient", ["#1DB954", "#191414"], N=100)

# Plot
plt.figure(figsize=(10, 6))
sns.heatmap(
    mood_feature_avg,
    annot=True, fmt='.2f',
    cmap=spotify_cmap,
    linewidths=0.5,
    linecolor='#191414',
    annot_kws={"color": "white", "fontSize": 10}
)

plt.title('Mood vs. Audio Feature Heatmap', fontsize=16, color='#1DB954', pad=20)
plt.xticks(rotation=45, ha='right', color='white')
plt.yticks(color='white')
plt.gca().set_facecolor('#191414')
plt.tight_layout()
plt.show()
```



```
import random
import math

df = df[df['mood'].isin(['happy', 'sad', 'energetic', 'calm'])]

print("Capturing your mood... pulling your vibe from Spotify.")

cap = cv2.VideoCapture(0, cv2.CAP_DSHOW)
if not cap.isOpened():
    print("Error: Could not open video capture")
    exit()

face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_frontalface_default.xml')

WINDOW_WIDTH, WINDOW_HEIGHT = 900, 600

# color palette
UI_COLORS = {
    'background': (5, 0, 10),          # Near-black with purple undertone
    'primary': (120, 40, 180),          # Deep purple (less vibrant)
    'secondary': (15, 5, 25),           # Darkest panel (almost black)
    'text_light': (220, 200, 240),      # Soft lavender (less bright)
    'text_muted': (100, 80, 120),       # Muted purple-gray
    'mood_happy': (255, 180, 50),        # Darker gold
    'mood_sad': (70, 130, 200),         # Deeper blue
    'mood_energetic': (220, 60, 130),    # Darker pink
    'mood_calm': (130, 90, 200)         # Deeper light purple
}
```

```

analysis_interval = 5
last_analysis = 0
current_emotion = "No face detected"
current_mood = None
last_played_url = None
confidence_threshold = 60

emotion_to_mood = {
    'happy': 'happy',
    'sad': 'sad',
    'disgust': 'energetic',
    'surprise': 'energetic',
    'fear': 'calm',
    'angry': 'calm',
    'neutral': 'neutral'
}

def play_mood_music(mood):
    global last_played_url
    if mood == 'neutral':
        return None
    mood_songs = df[df['mood'] == mood]
    if len(mood_songs) == 0:
        print(f"No songs found for mood: {mood}")
        return None
    available_songs = mood_songs[mood_songs['playable_url'] != last_played_url]
    if len(available_songs) == 0:
        available_songs = mood_songs
    selected_song = available_songs.sample(1).iloc[0]
    playable_url = selected_song['playable_url']
    webbrowser.open(playable_url, new=2)
    print(f"Now playing for mood: {mood}")
    last_played_url = playable_url
    return selected_song['uri'].split(':')[1]

def draw_text(img, text, pos, font_scale=0.7, color=UI_COLORS['text_light'], thickness=2):
    glow = (color[0] // 3, color[1] // 3, color[2] // 3)
    cv2.putText(img, text, (pos[0]+1, pos[1]+1), cv2.FONT_HERSHEY_SIMPLEX, font_scale, glow, thickness + 2, cv2.LINE_AA)
    cv2.putText(img, text, pos, cv2.FONT_HERSHEY_SIMPLEX, font_scale, color, thickness, cv2.LINE_AA)

def draw_mood_circle(img, mood, center, radius=30):
    colors = {
        'happy': UI_COLORS['mood_happy'],
        'sad': UI_COLORS['mood_sad'],
        'energetic': UI_COLORS['mood_energetic'],
        'calm': UI_COLORS['mood_calm'],
        'neutral': (100, 100, 100)
    }
    emojis = {
        'happy': ":)",
        'sad': ":",
        'energetic': ":D",
        'calm': ":|",
        'neutral': "://"
    }
    color = colors.get(mood, (100, 100, 100))
    cv2.circle(img, center, radius, color, -1)
    emoji_text = emojis.get(mood, "?")
    text_size = cv2.getTextSize(emoji_text, cv2.FONT_HERSHEY_SIMPLEX, 1.2, 3)[0]
    draw_text(img, emoji_text, (center[0] - text_size[0]//2, center[1] + text_size[1]//2),
              font_scale=1.2, color=(0, 0, 0), thickness=3)

def create_ui_frame():
    frame = np.zeros((WINDOW_HEIGHT, WINDOW_WIDTH, 3), dtype=np.uint8)
    frame[:] = UI_COLORS['background']

    # Pure black header with subtle purple edge
    header = frame.copy()
    cv2.rectangle(header, (0, 0), (WINDOW_WIDTH, 80), (10, 0, 20), -1) # Darkest purple-black

```

```

cv2.addWeighted(header, 0.95, frame, 0.05, 0, frame)

# Text with subtle glow
draw_text(frame, "MOODIFY", (30, 55),
          font_scale=1.8,
          color=UI_COLORS['primary'],
          thickness=3)
draw_text(frame, "emotion-driven music", (30, 85),
          font_scale=0.6,
          color=UI_COLORS['text_muted'],
          thickness=1)

# Footer - minimalist black bar with purple accent
cv2.rectangle(frame, (0, WINDOW_HEIGHT-40), (WINDOW_WIDTH, WINDOW_HEIGHT), (8, 0, 15), -1)
cv2.rectangle(frame, (0, WINDOW_HEIGHT-40), (WINDOW_WIDTH, WINDOW_HEIGHT-38), UI_COLORS['primary'], -1)

# Left-aligned footer text
draw_text(frame, "ESC to exit | Smile to change music",
          (20, WINDOW_HEIGHT-15),
          font_scale=0.6,
          color=UI_COLORS['text_light'],
          thickness=1)

return frame

def draw_waveform(ui_frame, height=50, color=UI_COLORS['primary'], animate=False):
    """Draw music wave above footer - animated when mood detected, static when neutral"""
    # Positioning - centered above footer with margins
    wave_area_height = 80 # Space reserved for waveform
    wave_y_center = WINDOW_HEIGHT - 40 - wave_area_height//2 # 40px footer height
    wave_x_start = 150 # Left margin
    wave_x_end = WINDOW_WIDTH - 150 # Right margin
    available_width = wave_x_end - wave_x_start

```

```

if animate:
    time_offset = time.time() * 2
else:
    time_offset = 0 # Static position

wave_points = []
for i in range(25): # Fewer bars for cleaner look
    if animate:
        # Musical wave pattern with animation
        main_wave = math.sin(i/1.5 + time_offset) * 0.7
        harmonic = math.cos(i/0.8 + time_offset*1.3) * 0.3
        wave_height = int(height * (main_wave + harmonic))
    else:
        # Static wave pattern
        wave_height = int(height * 0.3) # Small static wave
    wave_points.append(wave_height)

# Calculate bar dimensions
bar_count = len(wave_points)
bar_width = max(8, available_width // (bar_count * 2)) # Dynamic width
spacing = available_width // bar_count

# Draw the waveform
for i, h in enumerate(wave_points):
    x = wave_x_start + i * spacing
    bar_center = x + spacing//2

    # Gradient colored bar
    for w in range(bar_width):
        alpha = 0.6 + 0.4 * (abs(h)/height) # Height-based intensity
        fade_color = tuple(int(c * alpha) for c in color)
        cv2.line(ui_frame,
                  (bar_center - bar_width//2 + w, wave_y_center - h//2),
                  (bar_center - bar_width//2 + w, wave_y_center + h//2),
                  fade_color,

```

```

1)

# Highlight core of the bar
highlight_color = tuple(min(c + 60, 255) for c in color)
cv2.line(ui_frame,
         (bar_center, wave_y_center - h//2),
         (bar_center, wave_y_center + h//2),
         highlight_color,
         2)

# Add subtle baseline
cv2.line(ui_frame,
         (wave_x_start, wave_y_center),
         (wave_x_end, wave_y_center),
         (80, 40, 100), # Dark purple line
         1)

try:
    mood_detected = False
    waveform_animation_active = False

    while True:
        ui_frame = create_ui_frame()
        ret, frame = cap.read()
        if not ret:
            print("Failed to grab frame.")
            break
        frame = cv2.flip(frame, 1)
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        faces = face_cascade.detectMultiScale(gray, 1.3, 5)

        if len(faces) == 0:
            current_emotion = "No face detected"
            mood_detected = False
            waveform_animation_active = False

        for (x, y, w, h) in faces:
            cv2.rectangle(frame, (x, y), (x + w, y + h), UI_COLORS['primary'], 2)

            if time.time() - last_analysis > analysis_interval:
                try:
                    face_roi = frame[y:y + h, x:x + w]
                    result = DeepFace.analyze(face_roi, actions=['emotion'], enforce_detection=False,
                                              detector_backend='opencv', silent=True)
                    dominant_emotion = result[0]['dominant_emotion']
                    emotion_confidence = result[0]['emotion'][dominant_emotion]
                    current_emotion = f"{dominant_emotion} ({emotion_confidence:.1f}%)"
                    print(f"Detected: {current_emotion}")

                    if emotion_confidence >= confidence_threshold:
                        if dominant_emotion.lower() in emotion_to_mood:
                            mood_category = emotion_to_mood[dominant_emotion.lower()]
                            if mood_category != current_mood:
                                current_mood = mood_category
                                if mood_category != 'neutral':
                                    play_mood_music(current_mood)
                                    mood_detected = True
                                    waveform_animation_active = True
                        else:
                            mood_detected = False
                            waveform_animation_active = False
                    else:
                        mood_detected = False
                        waveform_animation_active = False
                except:
                    mood_detected = False
                    waveform_animation_active = False

            last_analysis = time.time()

```

```

finally:
    cap.release()
    cv2.destroyAllWindows()
    print("Come back soon for more mood matching beats =D")

    except Exception as e:
        print(f"DeepFace error: {str(e)[:100]}...")
        mood_detected = False
        waveform_animation_active = False

    display_mood = current_mood if mood_detected else "Detecting..."
    draw_text(frame, f"Mood: {display_mood}", (x, y - 10), font_scale=0.7, color=UI_COLORS['primary'], thickness=2)

    cam_w, cam_h = 480, 360
    resized_frame = cv2.resize(frame, (cam_w, cam_h))
    ui_frame[120:120 + cam_h, 50:50 + cam_w] = resized_frame

    # Right-side info panel
    info_panel = np.zeros((360, 340, 3), dtype=np.uint8)
    info_panel[:] = UI_COLORS['secondary']
    cv2.addWeighted(ui_frame[120:120 + 360, 550:550 + 340], 0.3, info_panel, 0.7, 0,
                    ui_frame[120:120 + 360, 550:550 + 340])

    draw_text(ui_frame, "Current Emotion:", (580, 160), font_scale=0.8, color=UI_COLORS['text_light'], thickness=2)
    draw_text(ui_frame, current_emotion, (580, 200), font_scale=0.7, color=UI_COLORS['primary'], thickness=2)

    draw_text(ui_frame, "Current Mood:", (580, 260), font_scale=0.8, color=UI_COLORS['text_light'], thickness=2)
    mood_text = current_mood.upper() if mood_detected and current_mood else "DETECTING..."
    mood_color = UI_COLORS.get('mood_' + current_mood, UI_COLORS['text_muted']) if mood_detected and current_mood else UI_COLORS['text_muted']
    draw_text(ui_frame, mood_text, (580, 300), font_scale=1.0, color=mood_color, thickness=3)

    if mood_detected or current_mood != 'neutral':
        draw_mood_circle(ui_frame, current_mood if current_mood else 'neutral', center=(750, 380), radius=50)

    # Draw the waveform - animate only when mood is detected and not neutral
    draw_waveform(ui_frame, animate=waveform_animation_active)

    cv2.imshow('Modify - Emotion Based Spotify Song Recommendations', ui_frame)
    if cv2.waitKey(1) == 27:
        break

```


Future Work

- Use larger, open song datasets (e.g., Last.fm, Spotify API)
- Improve mood classification with ensemble models
- Add speech recognition for audio mood analysis
- Build a mobile version of the app
- Add user login and preferences

Conclusion

Moodify blends computer vision, data science to create a dynamic, AI-based music experience. By analyzing facial expressions in real-time, it intelligently responds with curated songs based on the user's current mood. The project is supported by rich data visualizations, offering insights into audio feature patterns tied to moods. Moodify demonstrates the potential of emotional AI in media personalization and opens up future enhancements like voice interaction, mobile integration, and mood history analytics.

References

<https://www.kaggle.com/datasets/abdullahorzan/moodify-dataset?resource=download>

<https://pyvi.org/project/deepface/>

<https://medium.com/nerd-for-tech/deep-face-recognition-in-python-41522fb47028>

<https://www.geeksforgeeks.org/music-recommendation-system-using-machinelearning/>

<https://dev.to/gohashira/how-i-built-a-song-recommendation-system-with-python-scikit-learn-pandas-11ok>

